МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка баз данных»

Практическая работа № 3

Студент группы *ИВБО-20-24 Лияскин.В.А*

(подпись)

Ассистент *Шерри.Г.Д*

(подпись)

Отчет представлен «26» сентября 2026 г.

Москва 2026 г.

ЗАДАНИЕ 1. Использование оператора CASE

1.1 Категоризация данных с CASE

Задача: Категоризировать товары по цене (дешевые, средние, дорогие).

```sql
-- Категоризация товаров по ценовым категориям
SELECT
    name AS "Товар",
    retail_price AS "Цена, руб.",
    CASE
        WHEN retail_price < 80 THEN 'Дешевый товар'
        WHEN retail_price BETWEEN 80 AND 120 THEN 'Средняя цена'
        WHEN retail_price > 120 AND retail_price <= 200 THEN 'Дорогой товар'
        ELSE 'Премиум сегмент'
    END AS "Ценовая категория"
FROM product
ORDER BY retail_price;
```

product 1

SELECT name AS "Товар", retail_price AS | Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст | Запись

	Товар	Цена, руб.	Ценовая категория
1	Lipton Ice Tea 0.5л	75	Дешевый товар
2	Вода Святой Источник 0.5л	79,73	Дешевый товар
3	Pepsi 0.5л	80	Средняя цена
4	Fanta Апельсин 0.5л	82	Средняя цена
5	Sprite 0.5л	82	Средняя цена
6	Coca-Cola Classic 0.5л	85	Средняя цена
7	Молочный коктейль Ваниль	95	Средняя цена
8	Добрый Яблочный 1л	115	Средняя цена
9	Rich Апельсиновый 1л	120	Средняя цена
10	Monster Energy 0.5л	145	Дорогой товар
11	Red Bull 0.25л	150	Дорогой товар
12	Jacobs Кофе в зернах 250г	350	Премиум сегмент

Оператор CASE последовательно проверяет условия WHEN

Для каждого товара определяется ценовая категория

ELSE обрабатывает все случаи, не попавшие в предыдущие условия

1.2 Условная агрегация с CASE

Задача: Для каждого производителя посчитать количество дешевых и дорогих товаров.

```sql
-- Условная агрегация: подсчет товаров по категориям для каждого производителя
SELECT
    m.name AS "Производитель",
    COUNT(
        CASE
            WHEN p.retail_price < 100 THEN 1
            ELSE NULL
        END
    ) AS "Дешевые товары (<100 руб.)",
    COUNT(
        CASE
            WHEN p.retail_price >= 100 THEN 1
            ELSE NULL
        END
    ) AS "Дорогие товары (>=100 руб.)",
    COUNT(*) AS "Всего товаров"
FROM product p
JOIN manufacturer m ON p.manufacturer_id = m.id
GROUP BY m.id, m.name
ORDER BY "Всего товаров" DESC;
```

manufacturer 1

SELECT m.name AS "Производитель", CC | Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст

	Производитель	Дешевые товары (<100 руб.)	Дорогие товары (>=100 руб.)	Всего товаров
1	Coca-Cola Company	3	1	4
2	Red Bull	0	1	1
3	Jacobs	0	1	1
4	Monster Energy	0	1	1
5	PepsiCo	1	0	1
6	Балтмолпрод	1	0	1
7	Черноголовка	0	1	1
8	Lipton	1	0	1
9	Nestlé	1	0	1

CASE внутри агрегатной функции COUNT() позволяет условно подсчитывать строки

Когда условие выполняется, CASE возвращает 1 (который COUNT считает)

Когда условие не выполняется, возвращаем NULL (который COUNT игнорирует)

ЗАДАНИЕ 2. Использование подзапросов

2.1 Скалярный подзапрос

Задача: Найти все товары, цена которых выше средней цены по всему ассортименту.

```sql
-- Скалярный подзапрос: товары с ценой выше средней
SELECT
    name AS "Товар",
    retail_price AS "Цена, руб.",
    (SELECT ROUND(AVG(retail_price), 2) FROM product) AS "Средняя цена по магазину"
FROM product
WHERE retail_price > (SELECT AVG(retail_price) FROM product)
ORDER BY retail_price DESC;
```

product 1

SELECT name AS "Товар", retail_price AS | Введите SQL выражение чтобы отфильтровать результаты

Таблица

	A-Z Товар	123 Цена, руб.	123 Средняя цена по магазину
1	Jacobs Кофе в зернах 250г	350	121,56
2	Red Bull 0.25л	150	121,56
3	Monster Energy 0.5л	145	121,56

Скалярный подзапрос (SELECT AVG(retail_price) FROM product) возвращает одно значение (среднюю цену)

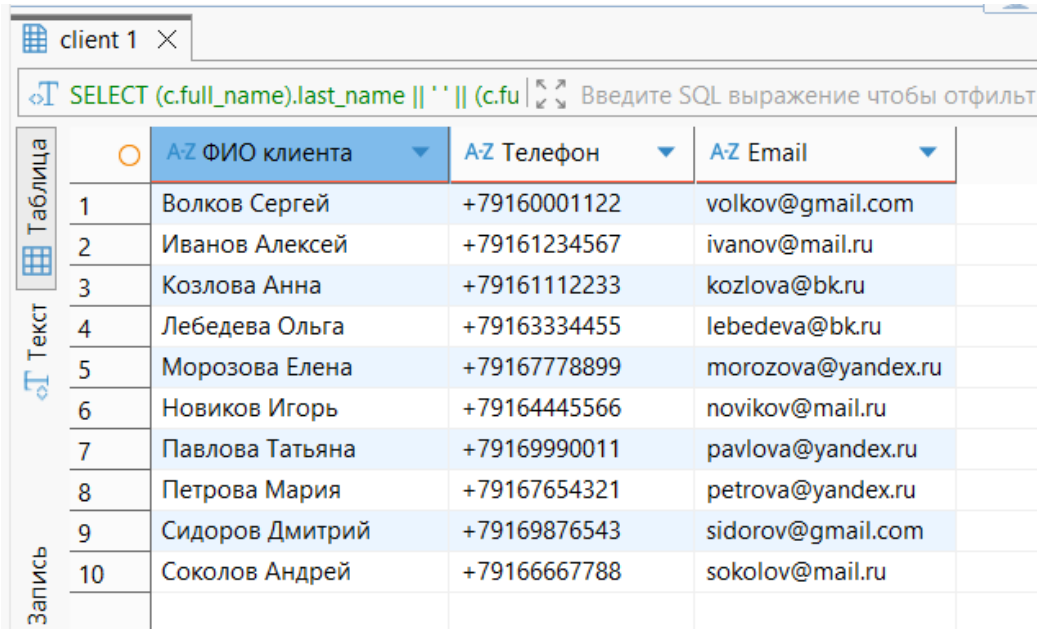
Подзапрос выполняется первым, вычисляя среднюю цену всех товаров

Внешний запрос фильтрует товары, у которых цена выше этого среднего значения

2.2 Многострочный подзапрос с IN

Задача: Найти клиентов, которые совершали покупки в январе 2024 года.

```sql
-- Многострочный подзапрос с IN: клиенты с покупками в январе 2024
SELECT
    (c.full_name).last_name || ' ' || (c.full_name).first_name AS "ФИО клиента",
    c.phone AS "Телефон",
    c.email AS "Email"
FROM client c
WHERE c.id IN (
    SELECT DISTINCT client_id
    FROM sale
    WHERE date >= '2024-01-01' AND date < '2024-02-01'
)
ORDER BY (c.full_name).last_name;
```



	ФИО клиента	Телефон	Email
1	Волков Сергей	+79160001122	volkov@gmail.com
2	Иванов Алексей	+79161234567	ivanov@mail.ru
3	Козлова Анна	+79161112233	kozlova@bk.ru
4	Лебедева Ольга	+79163334455	lebedeva@bk.ru
5	Морозова Елена	+79167778899	morozova@yandex.ru
6	Новиков Игорь	+79164445566	novikov@mail.ru
7	Павлова Татьяна	+79169990011	pavlova@yandex.ru
8	Петрова Мария	+79167654321	petrova@yandex.ru
9	Сидоров Дмитрий	+79169876543	sidorov@gmail.com
10	Соколов Андрей	+79166667788	sokolov@mail.ru

Подзапрос SELECT DISTINCT client_id FROM sale WHERE ... возвращает список ID клиентов (несколько строк, один столбец)

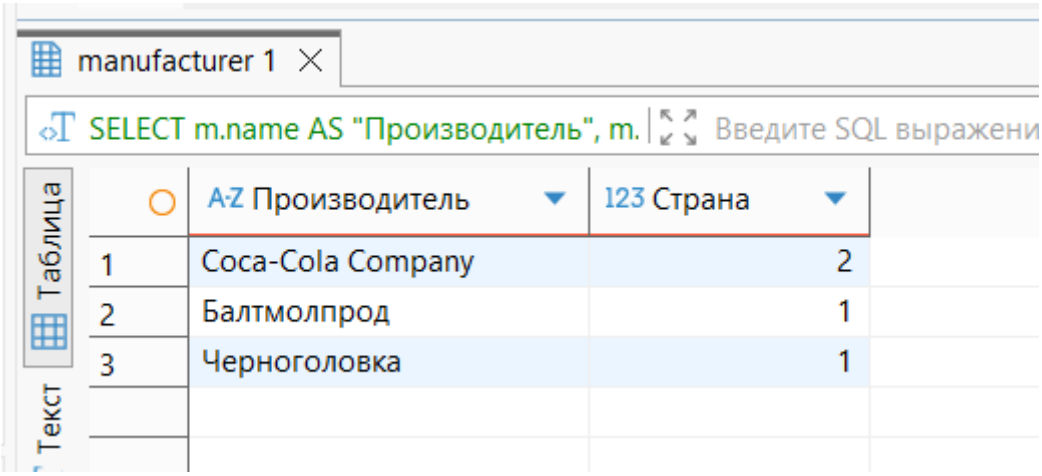
Оператор IN проверяет, содержится ли c.id в этом списке

DISTINCT убирает дубликаты (если клиент делал несколько покупок)

2.3 Коррелированный подзапрос с EXISTS

Задача: Найти производителей, которые выпускают товары с истекающим сроком годности (в течение 6 месяцев).

```sql
-- Коррелированный подзапрос с EXISTS: производители с товарами,
-- у которых скоро истекает срок годности
SELECT
    m.name AS "Производитель",
    m.country_id AS "Страна"
FROM manufacturer m
WHERE EXISTS (
    SELECT 1
    FROM product p
    WHERE p.manufacturer_id = m.id
    AND p.expiration_date IS NOT NULL
    AND p.expiration_date <= CURRENT_DATE + INTERVAL '6 months'
)
ORDER BY m.name;
```



Коррелированный (связанный) подзапрос ссылается на таблицу из внешнего запроса (m.id)

Для каждой строки производителя внешний запрос выполняет подзапрос

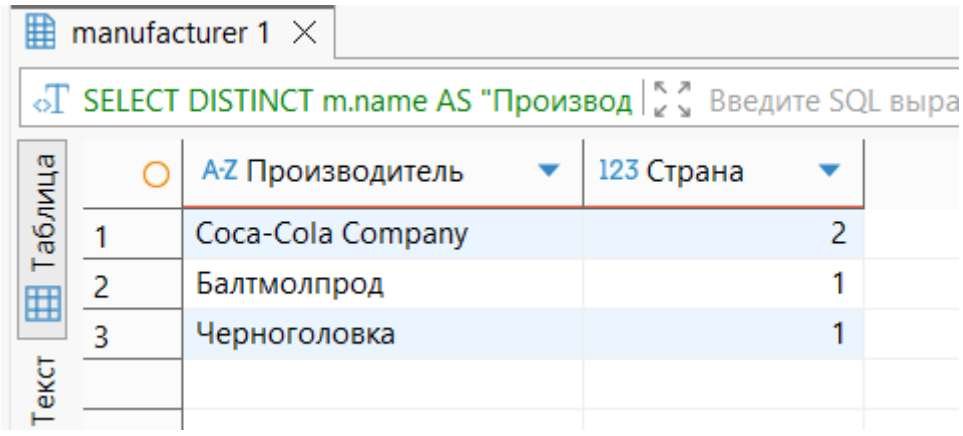
EXISTS проверяет, существует ли хотя бы один товар у этого производителя с истекающим сроком

Подзапрос возвращает TRUE/FALSE, а не данные

2.4 Альтернативное решение с JOIN

Задача: Решить предыдущую задачу через JOIN вместо EXISTS.

```sql
-- Альтернативное решение с JOIN
SELECT DISTINCT
    m.name AS "Производитель",
    m.country_id AS "Страна"
FROM manufacturer m
JOIN product p ON m.id = p.manufacturer_id
WHERE p.expiration_date IS NOT NULL
  AND p.expiration_date <= CURRENT_DATE + INTERVAL '6 months'
ORDER BY m.name;
```



JOIN физически соединяет таблицы manufacturer и product

WHERE фильтрует товары с истекающим сроком

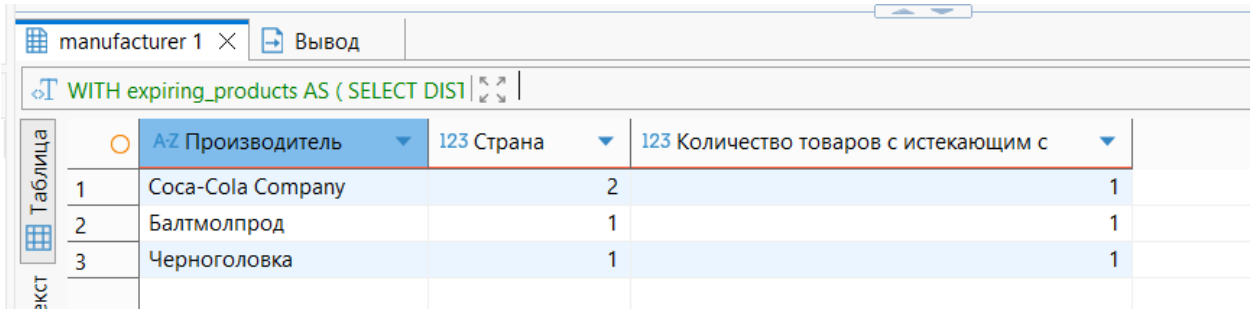
DISTINCT убирает дубликаты (если у производителя несколько таких товаров)

ЗАДАНИЕ 3. Использование CTE (Common Table Expressions)

3.1 Стандартное CTE

Задача: Переписать запрос из 2.3 с использованием CTE для лучшей читаемости.

```sql
-- Стандартное CTE: производители с товарами, у которых истекает срок годности
WITH expiring_products AS (
    -- Шаг 1: Находим все товары с истекающим сроком (в течение 6 месяцев)
    SELECT DISTINCT manufacturer_id
    FROM product
    WHERE expiration_date IS NOT NULL
      AND expiration_date <= CURRENT_DATE + INTERVAL '6 months'
)
-- Шаг 2: Используем CTE как временную таблицу
SELECT
    m.name AS "Производитель",
    m.country_id AS "Страна",
    COUNT(p.id) AS "Количество товаров с истекающим сроком"
FROM manufacturer m
JOIN expiring_products ep ON m.id = ep.manufacturer_id
JOIN product p ON m.id = p.manufacturer_id
    AND p.expiration_date <= CURRENT_DATE + INTERVAL '6 months'
GROUP BY m.id, m.name, m.country_id
ORDER BY m.name;
```



WITH expiring_products AS (...) создает именованное временное представление

CTE существует только на время выполнения одного запроса

3.2 Рекурсивное CTE

Задача: Создать иерархическую структуру категорий товаров и вывести всю иерархию.

Шаг 1: Добавляем поле parent_id в таблицу category

```sql
-- 1. Добавляем столбец parent_id для иерархии категорий
ALTER TABLE category
ADD COLUMN IF NOT EXISTS parent_id INT;

-- 2. Добавляем внешний ключ (ссылка на саму себя)
ALTER TABLE category
ADD CONSTRAINT IF NOT EXISTS fk_category_parent
FOREIGN KEY (parent_id)
REFERENCES category(id)
ON DELETE SET NULL;

-- 3. Все основные категории (id 1-10) делаем корневыми (parent_id = NULL)
UPDATE category
SET parent_id = NULL
WHERE id <= 10;

-- 4. Добавляем подкатегории для демонстрации иерархии
INSERT INTO category(name, parent_id) VALUES
('Кола', 1),
('Лимонад', 1),
('Яблочные соки', 2),
('Апельсиновые соки', 2)
ON CONFLICT DO NOTHING;

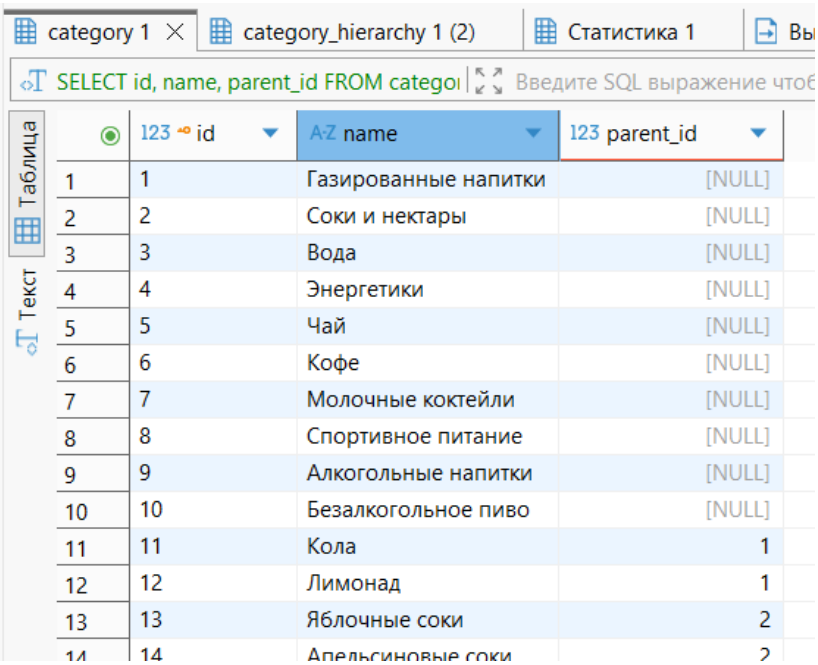
-- 5. Проверяем результат
SELECT id, name, parent_id
FROM category
ORDER BY id;
```

Шаг 2: Рекурсивный запрос

```sql
-- Рекурсивное CTE: вывод всей иерархии категорий
WITH RECURSIVE category_hierarchy AS (
    -- 1. Якорный запрос (базовый случай): корневые категории
    SELECT
        id,
        name,
        parent_id,
        0 AS level,
        CAST(name AS TEXT) AS full_path
    FROM category
    WHERE parent_id IS NULL

    UNION ALL

    -- 2. Рекурсивный запрос: подкатегории
    SELECT
        c.id,
        c.name,
        c.parent_id,
        ch.level + 1 AS level,
        ch.full_path || ' -> ' || c.name AS full_path
    FROM category c
    JOIN category_hierarchy ch ON c.parent_id = ch.id
)
-- 3. Финальный SELECT (ОБЯЗАТЕЛЕН!)
SELECT
    level AS "Уровень",
    REPEAT('  ', level) || name AS "Категория с отступами",
    full_path AS "Полный путь"
FROM category_hierarchy
ORDER BY full_path;
```



	123 id	A-Z name	123 parent_id
1	1	Газированные напитки	[NULL]
2	2	Соки и нектары	[NULL]
3	3	Вода	[NULL]
4	4	Энергетики	[NULL]
5	5	Чай	[NULL]
6	6	Кофе	[NULL]
7	7	Молочные коктейли	[NULL]
8	8	Спортивное питание	[NULL]
9	9	Алкогольные напитки	[NULL]
10	10	Безалкогольное пиво	[NULL]
11	11	Кола	1
12	12	Лимонад	1
13	13	Яблочные соки	2
14	14	Апельсиновые соки	2

1. Якорный запрос (Anchor Member):
    - Выполняется один раз в начале
    - Находит корневые элементы (где parent_id IS NULL)
    - Устанавливает начальный уровень level = 0
2. UNION ALL:
    - Объединяет результаты якоря и рекурсии
    - Важно: используется UNION ALL (не UNION), так как он быстрее (не проверяет дубликаты)
3. Рекурсивный запрос (Recursive Member):
    - Выполняется итеративно (многократно)
    - На каждой итерации находит подкатегории для результатов предыдущего шага
    - JOIN category_hierarchy ch — ссылка на сам CTE (рекурсия!)
    - Увеличивает уровень: level + 1
    - Строит полный путь: ch.full_path || ' -> ' || c.name
4. Условие остановки:
    - Рекурсия прекращается автоматически, когда JOIN не находит новых строк

ЗАДАНИЕ 4. Встроенные функции

4.1 Строковые функции

```sql
-- Демонстрация строковых функций
SELECT
    name AS "Исходное название",

    -- 1. LENGTH: длина строки (количество символов)
    LENGTH(name) AS "Длина названия",

    -- 2. UPPER: преобразование к верхнему регистру
    UPPER(name) AS "В ВЕРХНЕМ РЕГИСТРЕ",

    -- 3. LOWER: преобразование к нижнему регистру
    LOWER(name) AS "в нижнем регистре",

    -- 4. SUBSTRING: извлечение подстроки (первые 5 символов)
    SUBSTRING(name FROM 1 FOR 5) AS "Первые 5 символов",

    -- 5. CONCAT: объединение строк (название + единица измерения)
    CONCAT(name, ' (', (SELECT short_name FROM unit WHERE unit.id = product.unit_id LIMIT 1), ')')
        AS "Название с единицей",

    -- 6. TRIM: удаление пробелов по краям (на случай, если есть)
    TRIM(name) AS "Без пробелов"

FROM product
LIMIT 5;
```

product 1 | Вывод

SELECT name AS "Исходное название", | Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст

	Исходное название	Длина названия	В ВЕРХНЕМ РЕГИСТРЕ	в нижнем регистре	Первые 5 символов
1	Coca-Cola Classic 0.5л	22	COCA-COLA CLASSIC 0.5Л	coca-cola classic 0.5л	Coca-
2	Pepsi 0.5л	10	PEPSI 0.5Л	pepsi 0.5л	Pepsi
3	Fanta Апельсин 0.5л	19	FANTA АПЕЛЬСИН 0.5Л	fanta апельсин 0.5л	Fanta
4	Sprite 0.5л	11	SPRITE 0.5Л	sprite 0.5л	Sprit
5	Rich Апельсиновый 1л	20	RICH АПЕЛЬСИНОВЫЙ 1Л	rich апельсиновый 1л	Rich

чтобы отфильтровать результаты

Первые 5 символов	Название с единицей	Без пробелов
Coca-	Coca-Cola Classic 0.5л (мл)	Coca-Cola Classic 0.5л
Pepsi	Pepsi 0.5л (мл)	Pepsi 0.5л
Fanta	Fanta Апельсин 0.5л (мл)	Fanta Апельсин 0.5л
Sprit	Sprite 0.5л (мл)	Sprite 0.5л
Rich	Rich Апельсиновый 1л (л)	Rich Апельсиновый 1л

LENGTH(name) — полезен для валидации (проверка минимальной/максимальной длины)

UPPER/LOWER — для регистронезависимого поиска и сравнения

SUBSTRING — для извлечения части кода/названия

CONCAT — для форматирования вывода (можно использовать оператор || как альтернативу)

TRIM — для очистки данных от лишних пробелов

4.2 Функции работы с датой и временем

```sql
-- Демонстрация функций работы с датой
SELECT
    number AS "Номер поставки",
    date AS "Дата поставки",

    -- 1. CURRENT_DATE: текущая дата
    CURRENT_DATE AS "Сегодня",

    -- 2. AGE: разница между двумя датами (в человеко-понятном формате)
    AGE(CURRENT_TIMESTAMP, date) AS "Прошло времени",

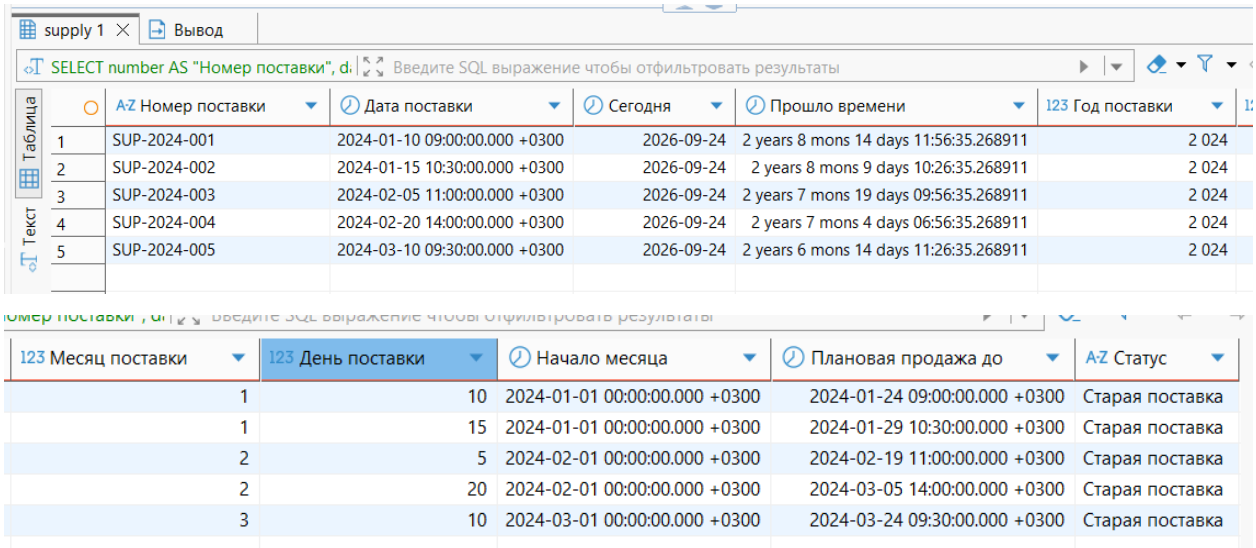
    -- 3. EXTRACT: извлечение части даты (год, месяц, день)
    EXTRACT(YEAR FROM date) AS "Год поставки",
    EXTRACT(MONTH FROM date) AS "Месяц поставки",
    EXTRACT(DAY FROM date) AS "День поставки",

    -- 4. DATE_TRUNC: усечение даты до начала периода
    DATE_TRUNC('month', date) AS "Начало месяца",

    -- 5. Расчет даты доставки (поставка + 14 дней)
    date + INTERVAL '14 days' AS "Плановая продажа до",

    -- 6. Проверка, просрочена ли поставка (более 30 дней назад)
    CASE
        WHEN date < CURRENT_TIMESTAMP - INTERVAL '30 days' THEN 'Старая поставка'
        ELSE 'Свежая поставка'
    END AS "Статус"

FROM supply
ORDER BY date
LIMIT 5;
```



supply 1 | Вывод

SELECT number AS "Номер поставки", d... Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст

	Номер поставки	Дата поставки	Сегодня	Прошло времени	Год поставки
1	SUP-2024-001	2024-01-10 09:00:00.000 +0300	2026-09-24	2 years 8 mons 14 days 11:56:35.268911	2 024
2	SUP-2024-002	2024-01-15 10:30:00.000 +0300	2026-09-24	2 years 8 mons 9 days 10:26:35.268911	2 024
3	SUP-2024-003	2024-02-05 11:00:00.000 +0300	2026-09-24	2 years 7 mons 19 days 09:56:35.268911	2 024
4	SUP-2024-004	2024-02-20 14:00:00.000 +0300	2026-09-24	2 years 7 mons 4 days 06:56:35.268911	2 024
5	SUP-2024-005	2024-03-10 09:30:00.000 +0300	2026-09-24	2 years 6 mons 14 days 11:26:35.268911	2 024

Месяц поставки	День поставки	Начало месяца	Плановая продажа до	Статус
1	10	2024-01-01 00:00:00.000 +0300	2024-01-24 09:00:00.000 +0300	Старая поставка
1	15	2024-01-01 00:00:00.000 +0300	2024-01-29 10:30:00.000 +0300	Старая поставка
2	5	2024-02-01 00:00:00.000 +0300	2024-02-19 11:00:00.000 +0300	Старая поставка
2	20	2024-02-01 00:00:00.000 +0300	2024-03-05 14:00:00.000 +0300	Старая поставка
3	10	2024-03-01 00:00:00.000 +0300	2024-03-24 09:30:00.000 +0300	Старая поставка

CURRENT_DATE — текущая дата (без времени)

AGE(date1, date2) — разница между датами в формате "X years Y mons Z days"

EXTRACT(part FROM date) — извлекает конкретную часть (YEAR, MONTH, DAY, HOUR и т.д.)

DATE_TRUNC('month', date) — усечение до начала месяца (2024-01-01 00:00:00)

4.3 Математические функции

```sql
-- Демонстрация математических функций
SELECT
    name AS "Товар",
    retail_price AS "Цена",

    -- 1. ROUND: округление до 2 знаков после запятой
    ROUND(retail_price, 2) AS "Округленная цена",

    -- 2. CEIL: округление до ближайшего целого ВВЕРХ
    CEIL(retail_price) AS "Цена (вверх)",

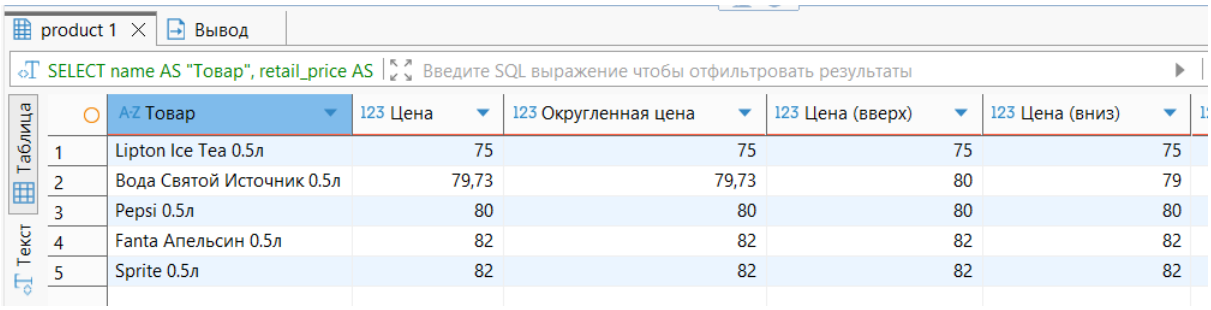
    -- 3. FLOOR: округление до ближайшего целого ВНИЗ
    FLOOR(retail_price) AS "Цена (вниз)",

    -- 4. ABS: модуль числа (абсолютное значение)
    ABS(retail_price - 100) AS "Отклонение от 100 руб.",

    -- 5. Расчет цены со скидкой 15% и округление
    ROUND(retail_price * 0.85, 2) AS "Цена со скидкой 15%",

    -- 6. POW: возведение в степень (цена в квадрате для примера)
    POW(retail_price, 2) AS "Цена²"

FROM product
ORDER BY retail_price
LIMIT 5;
```

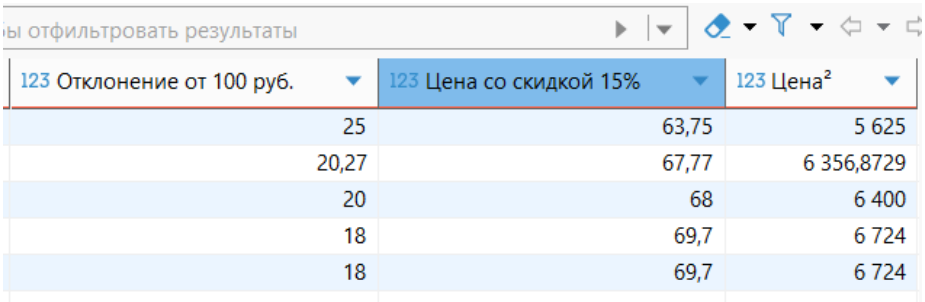


product 1 | Вывод

SELECT name AS "Товар", retail_price AS | Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст

	Товар	Цена	Округленная цена	Цена (вверх)	Цена (вниз)
1	Lipton Ice Tea 0.5л	75	75	75	75
2	Вода Святой Источник 0.5л	79,73	79,73	80	79
3	Pepsi 0.5л	80	80	80	80
4	Fanta Апельсин 0.5л	82	82	82	82
5	Sprite 0.5л	82	82	82	82



ы отфильтровать результаты

Отклонение от 100 руб.	Цена со скидкой 15%	Цена²
25	63,75	5 625
20,27	67,77	6 356,8729
20	68	6 400
18	69,7	6 724
18	69,7	6 724

ROUND(num, decimals) — математическое округление (для финансовых расчетов)

CEIL(num) — всегда округляет вверх (полезно для расчета минимального количества упаковок)

FLOOR(num) — всегда округляет вниз (для расчета максимального количества)

ABS(num) — модуль числа (всегда положительное значение)

ЗАДАНИЕ 5. Работа с JSONB

5.1 Подготовка: создание таблицы логов

```sql
-- Создаем таблицу для хранения логов событий в формате JSONB
CREATE TABLE IF NOT EXISTS user_logs (
    log_id SERIAL PRIMARY KEY,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    user_id INT,
    client_id INT,
    event_data JSONB  -- Поле для хранения неструктурированных данных
);

COMMENT ON TABLE user_logs IS 'Таблица логов событий системы (продажи, входы, действия)';
```

JSONB — бинарный формат JSON в PostgreSQL (эффективное хранение и индексирование)

В отличие от типа JSON, JSONB:

Хранится в разобранном виде (быстрее читается)

Поддерживает индексацию (GIN индексы)

Удаляет дубликаты ключей

Не сохраняет порядок ключей и пробелы

5.2 Наполнение таблицы тестовыми данными

```sql
-- Генерация событий типа "sale" (продажа)
INSERT INTO user_logs(created_at, user_id, client_id, event_data)
VALUES
-- Событие 1: продажа Coca-Cola
(NOW() - INTERVAL '1 day', 1, 1,
 '{"event": "sale", "store_id": 1, "items": [
     {"product_id": 1, "product_name": "Coca-Cola Classic 0.5л", "quantity": 2, "price": 85.00},
     {"product_id": 7, "product_name": "Вода Святой Источник 0.5л", "quantity": 3, "price": 45.00}
 ], "total_amount": 305.00, "payment_method": "card", "success": true}'::jsonb),

-- Событие 2: продажа Pepsi
(NOW() - INTERVAL '2 days', 2, 2,
 '{"event": "sale", "store_id": 1, "items": [
     {"product_id": 2, "product_name": "Pepsi 0.5л", "quantity": 4, "price": 80.00}
 ], "total_amount": 320.00, "payment_method": "cash", "success": true}'::jsonb),

-- Событие 3: продажа Red Bull
(NOW() - INTERVAL '3 days', 1, 3,
 '{"event": "sale", "store_id": 2, "items": [
     {"product_id": 8, "product_name": "Red Bull 0.25л", "quantity": 2, "price": 150.00},
     {"product_id": 9, "product_name": "Monster Energy 0.5л", "quantity": 1, "price": 145.00}
 ], "total_amount": 445.00, "payment_method": "card", "success": true}'::jsonb),

-- Событие 4: вход сотрудника в систему
(NOW() - INTERVAL '1 hour', 6, NULL,
 '{"event": "login", "employee_id": 6, "ip_address": "192.168.1.100", "browser": "Chrome", "os": "Windows", "success": true}'::

-- Событие 5: ошибка при продаже
(NOW() - INTERVAL '5 days', 2, 4,
 '{"event": "sale", "store_id": 1, "items": [], "total_amount": 0, "payment_method": "card", "success": false, "error": "Недост

-- Событие 6: еще одна продажа
(NOW() - INTERVAL '10 days', 1, 5,
 '{"event": "sale", "store_id": 3, "items": [
     {"product_id": 11, "product_name": "Jacobs Кофе в зернах 250г", "quantity": 1, "price": 350.00}
 ], "total_amount": 350.00, "payment_method": "contactless", "success": true}'::jsonb),

-- Событие 7: вход другого сотрудника
(NOW() - INTERVAL '30 minutes', 5, NULL,
 '{"event": "login", "employee_id": 5, "ip_address": "192.168.1.105", "browser": "Firefox", "os": "Linux", "success": true}'::j
```

Используем ::jsonb для явного приведения к типу JSONB

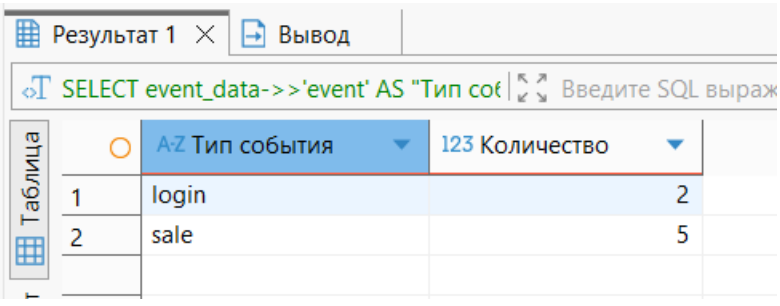
Храним разную структуру данных в одном поле:

Для продаж: массив товаров items, сумма, способ оплаты

Для входов: IP-адрес, браузер, ОС

5.3 Проверка данных

```sql
-- Проверяем, сколько записей каждого типа событий
SELECT
    event_data->>'event' AS "Тип события",
    COUNT(*) AS "Количество"
FROM user_logs
GROUP BY event_data->>'event';
```



event_data->>'event' — оператор ->> извлекает значение по ключу как текст

5.4 Извлечение и фильтрация JSONB

Задача 1: Найти все продажи с суммой чека больше 300 рублей.

```sql
-- Фильтрация по значению в JSONB: дорогие продажи
SELECT
    log_id,
    created_at,
    client_id,
    event_data->>'event' AS "Событие",
    (event_data->>'total_amount')::NUMERIC AS "Сумма, руб.",
    event_data->>'payment_method' AS "Способ оплаты"
FROM user_logs
WHERE event_data->>'event' = 'sale'
  AND (event_data->>'total_amount')::NUMERIC > 300
ORDER BY (event_data->>'total_amount')::NUMERIC DESC;
```

user_logs 1 | Вывод

SELECT log_id, created_at, client_id, event | Введите SQL выражение чтобы отфильтровать результаты

	log_id	created_at	client_id	Событие	Сумма, руб.	Способ оплаты
1	3	2026-09-21 21:01:25.146 +0300	3	sale	445	card
2	6	2026-09-14 21:01:25.146 +0300	5	sale	350	contactless
3	2	2026-09-22 21:01:25.146 +0300	2	sale	320	cash
4	1	2026-09-23 21:01:25.146 +0300	1	sale	305	card

->> извлекает значение как текст: event_data->>'total_amount' → "305.00"

::NUMERIC — приводим текст к числу для сравнения

->>'event' = 'sale' — фильтруем только продажи

Задача 2: Найти все успешные продажи, оплаченные картой.

```sql
-- Фильтрация по нескольким полям JSONB
SELECT
    log_id,
    created_at,
    client_id,
    event_data->>'payment_method' AS "Способ оплаты",
    (event_data->>'total_amount')::NUMERIC AS "Сумма"
FROM user_logs
WHERE event_data @> '{"event": "sale", "payment_method": "card", "success": true}'::jsonb
ORDER BY created_at;
```

user_logs 1 | Вывод

SELECT log_id, created_at, client_id, event | Введите SQL выражение чтобы отфильтровать результаты

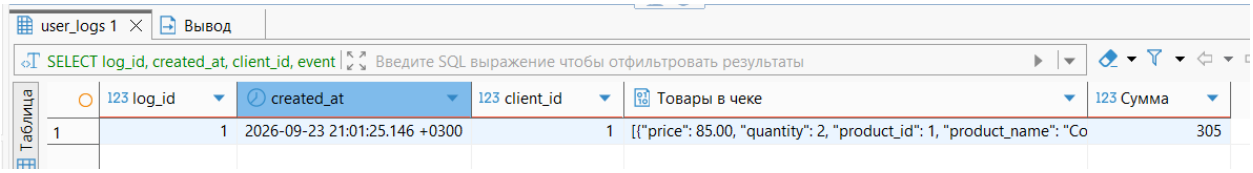
	log_id	created_at	client_id	Способ оплаты	Сумма
1	3	2026-09-21 21:01:25.146 +0300	3	card	445
2	1	2026-09-23 21:01:25.146 +0300	1	card	305

Оператор @> проверяет вхождение (containment)

'{"event": "sale", ...}'::jsonb — проверяет, содержит ли event_data указанные ключи и значения

Задача 3: Найти продажи конкретного товара (Coca-Cola, product_id = 1).

```sql
-- Поиск по вложенному массиву items
SELECT
    log_id,
    created_at,
    client_id,
    event_data->'items' AS "Товары в чеке",
    (event_data->>'total_amount')::NUMERIC AS "Сумма"
FROM user_logs
WHERE event_data->>'event' = 'sale'
  AND event_data->'items' @> '[{"product_id": 1}]'::jsonb
ORDER BY created_at;
```



event_data->'items' — извлекаем массив товаров как JSONB

@> '[{"product_id": 1}]' — проверяем, содержит ли массив объект с product_id: 1

Для массивов в JSONB нужно указывать квадратные скобки [...]

5.5 Агрегация данных из JSONB

Задача: Посчитать общую выручку и среднюю сумму чека по всем продажам.

```sql
-- Агрегация данных из JSONB
SELECT
    COUNT(*) AS "Количество продаж",
    SUM((event_data->>'total_amount')::NUMERIC) AS "Общая выручка, руб.",
    ROUND(AVG((event_data->>'total_amount')::NUMERIC), 2) AS "Средний чек, руб.",
    MIN((event_data->>'total_amount')::NUMERIC) AS "Минимальный чек",
    MAX((event_data->>'total_amount')::NUMERIC) AS "Максимальный чек"
FROM user_logs
WHERE event_data->>'event' = 'sale'
  AND event_data->>'success' = 'true';
```

Результат 1 | Вывод

SELECT COUNT(*) AS "Количество прода | Введите SQL выражение чтобы отфильтровать результаты

Таблица

	123 Количество продаж	123 Общая выручка, руб.	123 Средний чек, руб.	123 Минимальный чек	123 Максимальный чек
1	4	1 420	355	305	445

Извлекаем total_amount из JSON, приводим к NUMERIC

Применяем агрегатные функции: SUM, AVG, MIN, MAX

5.6 Модификация JSONB

Задача 1: Массово добавить поле "source": "web" ко всем логам.

```sql
-- Массовое обновление: добавление нового поля во все JSON-документы
UPDATE user_logs
SET event_data = event_data || '{"source": "web"}'::jsonb;

-- Проверяем результат
SELECT
    log_id,
    event_data->>'event' AS "Событие",
    event_data->>'source' AS "Источник"
FROM user_logs
LIMIT 5;
```

user_logs 1 | Статистика 1 | Вывод

Таблица

Текст

	123 log_id	A-Z Событие	A-Z Источник
1	1	sale	web
2	2	sale	web
3	3	sale	web
4	4	login	web
5	5	sale	web

Оператор || — конкатенация (объединение) JSONB объектов

Если ключ уже существует — он перезаписывается

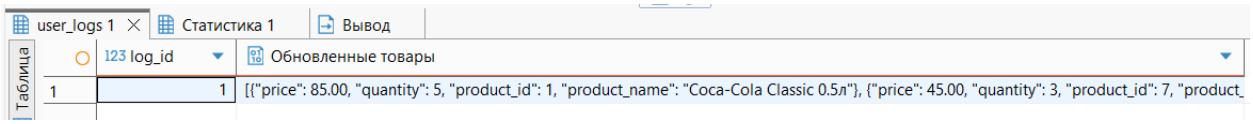
Если ключа нет — он добавляется

Обновление применяется ко всем строкам (нет WHERE)

Задача 2: Точечно изменить количество товара в конкретной продаже.

```sql
-- Точечное изменение вложенного значения с jsonb_set
UPDATE user_logs
SET event_data = jsonb_set(
    event_data,
    '{items, 0, quantity}',  -- Путь: items[0].quantity
    '5'::jsonb  -- Новое значение (обязательно в формате jsonb!)
)
WHERE log_id = 1
  AND event_data->>'event' = 'sale';

-- Проверяем изменение
SELECT
    log_id,
    event_data->'items' AS "Обновленные товары"
FROM user_logs
WHERE log_id = 1;
```



jsonb_set(target, path, new_value) — функция точечной замены

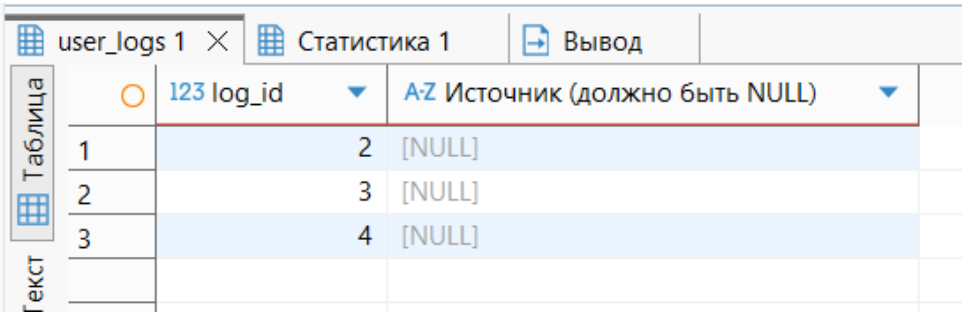
path — массив строк с путем к полю: '{items, 0, quantity}' означает items[0].quantity

new_value — новое значение в формате JSONB ('5'::jsonb, а не просто 5)

Задача 3: Удалить поле из JSON (например, source после тестирования).

```sql
-- Удаление поля из JSONB
UPDATE user_logs
SET event_data = event_data - 'source'
WHERE event_data ? 'source';

-- Проверяем, что поле удалено
SELECT
    log_id,
    event_data->>'source' AS "Источник (должно быть NULL)"
FROM user_logs
LIMIT 3;
```



Оператор - удаляет ключ из JSONB

event_data ? 'source' — проверяет существование ключа (возвращает TRUE/FALSE)

WHERE гарантирует, что обновляем только строки, где поле есть